Gestión de talleres

Integrantes y Roles

Rol	Integrante	Responsabilidad
Coordinador	Leonardo Coronel	Controla el tiempo de cada fase y verifica que todos participen.
Programador	Mateo Navas	Escribe e integra el código: funciones, filtros con where y el menú de consola.
Apoyo en pruebas y documentación	David Cocha	Revisa la regla de negocio, apoya al tester y redacta las respuestas del documento.
Analista	Esteven Viracocha	Define las estructuras de datos (List, Map, Set) y revisa que tengan sentido.
Tester	Isak Gomez	Ejecuta los casos de prueba y registra los resultados esperados y reales.
Vocera	Paola Narváez	Presenta la solución y la demostración final (2–3 minutos).

1. Actividades obligatorias

A. Variables y tipos

```
PS C:\Users\misak\OneDrive\Documentos\UNIBE\ProgramacionV\Flutter\Clase1\PV_G2_ACTIVITIES> dart run .\SECOND_ACTIVITY\gestion_talleres\actividad_A.dart
--- Actividad A: Variables y tipos ---
Universidad (String): UNIBE
Taller (String): Flutter inicial
Cupo del taller (int): 20
Inscripciones abiertas (bool): true

¿Qué representa cada tipo?
  String -> texto: aquí, el nombre de la universidad y del taller.
  int    -> número entero: aquí, la cantidad de cupos del taller.
  bool   -> verdadero o falso: aquí, si las inscripciones están abiertas.

Talleres en la lista: 4
Participantes únicos en el Set: 3
```

B. List, Map y Set

```
---Actividad B: List, Map y Set ---
Talleres almacenados en la lista:
  - Flutter inicial
  - Dart desde cero
  - Diseño UX
  - Flutter inicial
Cupo de Flutter inicial (Map): 20
Cantidad de talleres en la lista: 4
Participantes únicos en el Set: 3
¿Por qué la List tiene repetidos y el Set no?
  La List guarda cada elemento en su posición, aunque se repita (por eso "Flutter inicial" aparece 2 veces).
  El Set solo guarda valores únicos y descarta los que ya contiene (por eso "Ana" cuenta 1 sola vez).
```

C. Funciones y parámetros

```
--- Talleres de UNIBE ---

--- Actividad C: Funciones y parámetros ---
Cupo de "Flutter inicial": 20
Cupo de "Robótica" (no existe): 0
Abiertas=true,  cupos=5 -> true
Abiertas=true,  cupos=0 -> false
Abiertas=false, cupos=5 -> false
Abiertas=false, cupos=0 -> false
Taller: Dart desde cero | Cupos disponibles: 25
Dart desde cero
Taller: Diseño UX | Cupos disponibles: 15

Preguntas:
1) ¿Qué función devuelve un valor?
   obtenerCupo (int), puedeInscribirse (bool) y crearResumen (String). mostrarEncabezado es void: solo imprime y no devuelve nada.
2) ¿Diferencia entre parámetro posicional y nombrado?
   El posicional se identifica por su orden: obtenerCupo(mapa, taller). El nombrado va entre llaves {} y se pasa con su nombre: crearResumen(taller: ..., cupos: ...), sin importar el orden.
3) ¿Para qué se usa required?
   Obliga a pasar ese parámetro nombrado al llamar la función; si se omite, el código no compila.
```

D. Iterables y filtrado

```
---Actividad D: Iterables y filtrado ---
Talleres que contienen "Flutter": [Flutter inicial, Flutter inicial]
Cantidad de resultados: 2
Talleres con más de 10 caracteres: [Flutter inicial, Dart desde cero, Flutter inicial]
Cantidad de resultados: 3

Preguntas:
1) ¿Qué hace where?
   Recorre la colección y conserva solo los elementos para los que la condición devuelve true. No modifica la lista original.
2) ¿Por qué la variable puede ser Iterable<String>?
   Porque where devuelve un Iterable<String>, no una List. List es un tipo de Iterable, y el resultado se evalúa de forma perezosa: se calcula al recorrerlo o al llamar toList().
```

E. Regla de negocio

```
--- Actividad E: Regla de negocio ---
Participantes al inicio: {Ana, Luis, Marta}

1) Registro válido (Carlos):
   Éxito: Carlos fue registrado/a. Total de participantes: 4.
2) Persona ya registrada (Ana):
   Rechazado: Ana ya está registrado/a.
3) Inscripciones cerradas (Sofía):
   Rechazado: las inscripciones están cerradas.
4) Sin cupos disponibles (Pedro):
   Rechazado: no hay cupos disponibles.
5) Nombre vacío:
   Rechazado: el nombre no puede estar vacío.

Participantes al final: {Ana, Luis, Marta, Carlos}
```

F. Integración y presentación

```
--- Talleres de UNIBE ---

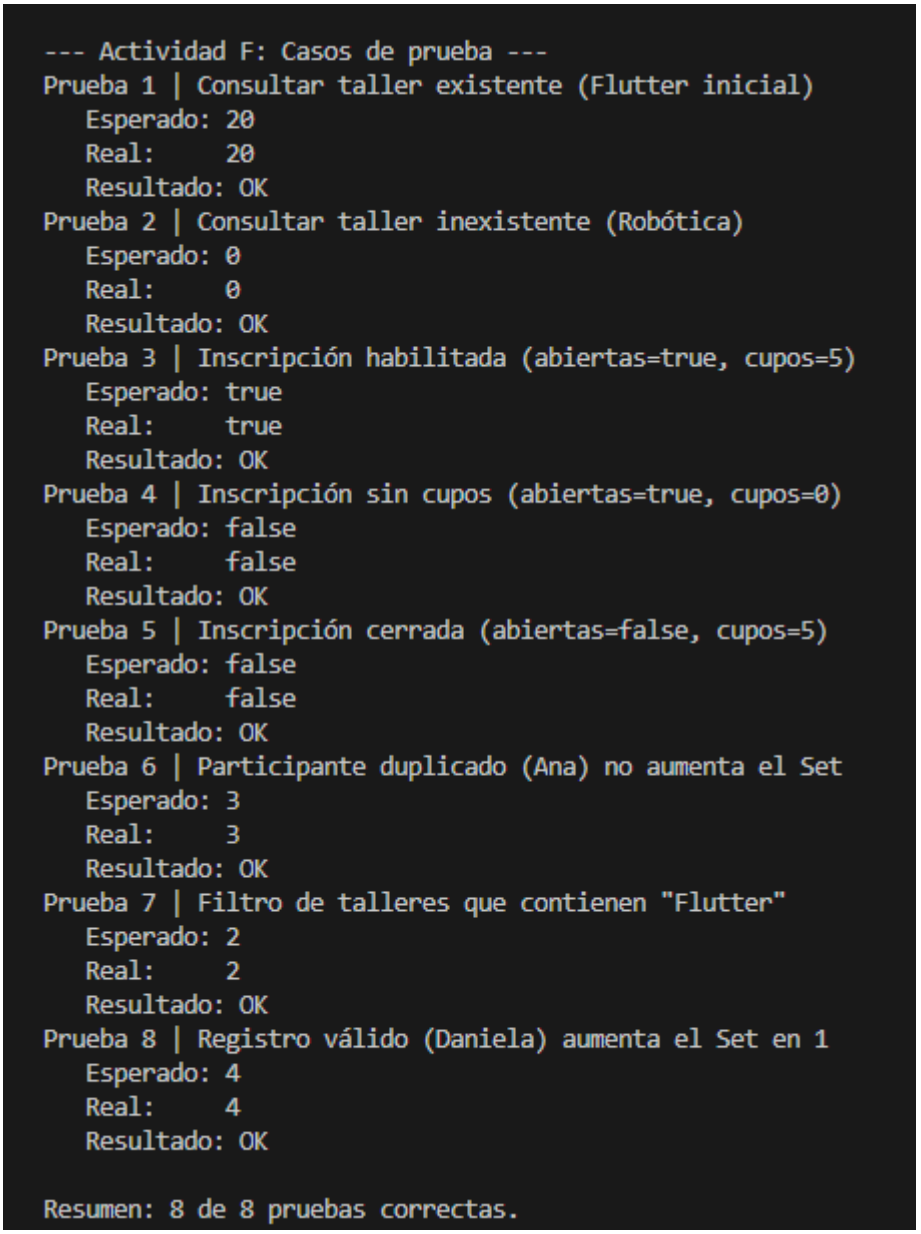
--- Actividad A: Variables y tipos ---
Universidad (String): UNIBE
Taller (String): Flutter inicial
Cupo del taller (int): 20
Inscripciones abiertas (bool): true
¿Qué representa cada tipo?
  String -> texto: aquí, el nombre de la universidad y del taller.
  int    -> número entero: aquí, la cantidad de cupos del taller.
  bool   -> verdadero o falso: aquí, si las inscripciones están abiertas.

--- Actividad B: List, Map y Set ---
Talleres almacenados en la lista:
  - Flutter inicial
  - Dart desde cero
  - Diseño UX
  - Flutter inicial
Cupo de Flutter inicial (Map): 20
Cantidad de talleres en la lista: 4
Participantes únicos en el Set: 3
¿Por qué la List tiene repetidos y el Set no?
  La List guarda cada elemento en su posición, aunque se repita (por eso "Flutter inicial" aparece 2 veces).
  El Set solo guarda valores únicos y descarta los que ya contiene (por eso "Ana" cuenta 1 sola vez).

--- Actividad C: Funciones y parámetros ---
Cupo de "Flutter inicial": 20
Cupo de "Robótica" (no existe): 0
Abiertas=true,  cupos=5 -> true
Abiertas=true,  cupos=0 -> false
Abiertas=false, cupos=5 -> false
Abiertas=false, cupos=0 -> false
Taller: Dart desde cero | Cupos disponibles: 25
Dart desde cero
Devuelven valor: obtenerCupo, puedeInscribirse y crearResumen; mostrarEncabezado es void.
Posicional: se identifica por su orden. Nombrado: va entre {} y se pasa con su nombre. required obliga a pasar el parámetro nombrado.

--- Actividad D: Iterables y filtrado ---
Talleres que contienen "Flutter": [Flutter inicial, Flutter inicial]
Cantidad de resultados (Flutter): 2
Talleres con más de 10 caracteres: [Flutter inicial, Dart desde cero, Flutter inicial]
Cantidad de resultados (más de 10 caracteres): 3
where conserva solo los elementos que cumplen la condición y devuelve un Iterable<String> (se calcula al recorrerlo o al usar toList()).
La variable puede ser Iterable<String> porque where devuelve un Iterable, no una List; List es un tipo de Iterable.

--- Actividad E: Regla de negocio ---
Éxito: Carlos fue registrado/a. Total de participantes: 4.
Rechazado: Ana ya está registrado/a.
Rechazado: las inscripciones están cerradas.
Rechazado: no hay cupos disponibles.
Participantes al final: {Ana, Luis, Marta, Carlos}
```



2. Tabla de casos de prueba

Las pruebas se ejecutaron con `dart run` y el programa terminó con el resumen: 8 de 8 pruebas correctas.

N.º	Caso	Entrada	Resultado esperado	real	Estado
1	Consultar taller existente	`'Flutter inicial'`	Devuelve 20 cupos	20	OK
2	Consultar taller inexistente	`'Robótica'`	Devuelve 0	0	OK
3	Inscripción habilitada	abiertas = true, cupos = 5	Sí puede inscribirse (`true`)	`true`	OK
4	Inscripción sin cupos	abiertas = true, cupos = 0	No puede inscribirse (`false`)	`false`	OK
5	Inscripción cerrada	abiertas = false,	No puede inscribirse (`false`)	`false`	OK

		cupos = 5			
6	Participante duplicado	`'Ana'`	No aumenta el tamaño del Set (se mantiene en 3)	3	OK
7	Filtro de talleres	contiene `'Flutter'`	Solo devuelve los talleres que cumplen el criterio (2)	2	OK
8	Registro válido (prueba adicional)	`'Daniela'`	El Set aumenta en 1 (pasa de 3 a 4)	4	OK

N.º	Caso	Entrada	Resultado esperado	real	Estado	Observación
1	Consultar taller existente	`'Flutter inicial'`	Devuelve 20 cupos	20	OK	Se ejecutó obtenerCupo con parámetros: Map cuposPorTaller y Flutter inicial , esta función retorna el index del Map por el nombre de búsqueda o default 0, en este caso Retorna los cupos exitosamente la función es Case sensitive asi que flutter inicial no encuentra resultados
2	Consultar taller inexistente	`'Robótica'`	Devuelve 0	0	OK	Se ejecutó obtenerCupo con parámetros: Map cuposPorTaller y Robótica , esta función retorna el index del Map por el nombre de búsqueda o default 0, en este caso retorna 0 ya que no existe el index Robótica en el Map
3	Inscripción habilitada	abiertas = true, cupos = 5	Sí puede inscribirse (`true`)	`true`	OK	Se ejecuta puedeInscribirse con parámetros true y 5 cupos la función evalúa si parámetro a es verdadero y parámetro b es mayor a 0, por lo tanto el resultado es verdadero
4	Inscripción sin cupos	abiertas = true, cupos = 0	No puede inscribirse (`false`)	`false`	OK	Se ejecuta puedeInscribirse con parámetros false y 0 cupos la función evalúa si parámetro a es verdadero y parámetro b es mayor a 0, por lo tanto el resultado es falso ya que ambos parámetros no cumplen las condiciones
5	Inscripción cerrada	abiertas = false, cupos = 5	No puede inscribirse (`false`)	`false`	OK	Se ejecuta puedeInscribirse con parámetros false y 5

						cupos la función evalúa si parámetro a es verdadero y parámetro b es mayor a 0, por lo tanto el resultado es falso ya que parámetro a es false
6	Participante duplicado	`'Ana'`	No aumenta el tamaño del Set (se mantiene en 3)	3	OK	Se ejecuta registrarParticipante con parámetros Ana, Set participantesPrueba, true (si hay inscripciones) y 5 cupos disponibles, la función evalúa que exista un nombre, se permitan inscripciones, número de cupos no sea menor o igual a 0 y por último que el Set no contenga un nombre exactamente igual al proporcionado, por lo tanto en este caso se obtiene como resultado que no se inscriba el estudiante ya que existe previamente en el Set.
7	Filtro de talleres	contiene `'Flutter'`	Solo devuelve los talleres que cumplen el criterio (2)	2	OK	Se ejecuta un pipe donde filtra List de talleres por coincidencia de contenido, el filtro es case sensitive por lo tanto se debe buscar por una coincidencia exacta, en este caso se observa en el resultado dos casos donde Flutter fue encontrado en el contenido
8	Registro válido (prueba adicional)	`'Daniela'`	El Set aumenta en 1 (pasa de 3 a 4)	4	OK	Se ejecuta registrarParticipante con parámetros Daniela, Set participantesPrueba, true (si hay inscripciones) y 5 cupos disponibles, la función evalúa que exista un nombre, se permitan inscripciones, número de cupos no sea menor o igual a 0 y por último que el Set no contenga un nombre exactamente igual al proporcionado, por lo tanto en este caso se obtiene como resultado que el Set contiene a la nueva estudiante Daniela.

2. Preguntas de reflexión

1. ¿Qué diferencia práctica existe entre List y Set?

La `List` es una colección ordenada que se accede por posición y acepta repetidos. El `Set` no admite repetidos y no se accede por índice. En el programa, la `List` guarda los talleres (con "Flutter inicial" dos veces) y el `Set` guarda los participantes, donde "Ana" cuenta una sola vez. El `Set` es ideal para preguntar rápido si alguien ya está con contains.

2. ¿Cuándo resulta útil un Map en una aplicación móvil?

Cuando cada dato se busca por un nombre o clave y no por posición. Aquí asocia cada taller con su cupo. En una app móvil sirve para precios por producto, ajustes de usuario o traducciones, y los datos JSON que llegan de una API se leen como map.

3. ¿Qué ventaja tiene encapsular una operación en una función en lugar de repetir código?

Se escribe la lógica una sola vez y se reutiliza. Si cambia la regla, se corrige en un solo lugar, el código se lee mejor y es más fácil de probar. Por ejemplo, puedeInscribirse se usó en varias pruebas sin repetir la condición.

4. ¿Qué ocurre cuando una función devuelve un valor y qué ocurre cuando su tipo de retorno es void?

Si la función devuelve un valor, se puede guardar o usar en otra expresión, como int cupo = onbtenerCupo. Si es void, solo ejecuta una acción, por ejemplo imprimir, y no hay resultado que guardar.

5. ¿Por qué puede ser útil declarar una variable como Iterable en lugar de convertir inmediatamente el resultado a List?

Where no crea una lista nueva, calcula los elementos cuando se recorren. Eso ahorra memoria con colecciones grandes y permite encadenar varios filtros, convirtiendo a `List` solo al final y si hace falta. Además, where ya devuelve un Iterable.

6. ¿Qué validaciones agregarían si esta lógica se conectara a una pantalla de registro real?

Nombre no vacío y con longitud y caracteres razonables, y normalizar mayúsculas y espacios para que "Ana" y "ana" no cuentan como dos personas. Identificar al participante por correo o cédula en vez de por nombre, comprobar que el taller exista, descontar el cupo de ese taller al registrar, y mostrar mensajes claros en pantalla. Con datos guardados en un servidor, habría que cuidar también que dos registros simultáneos no ocupen el último cupo a la vez.